

Canny Edge Detector

Relazione di ripasso completa

Documento strutturato per ripassare teoria, formule e passaggi implementativi dell'algoritmo sviluppato.

Contenuto	descrizione
Obiettivo	ricostruire tutte le fasi del progetto con formule, significato geometrico e note implementative
Focus	kernel gaussiani, convoluzione, gradiente, NMS, doppia soglia, isteresi
Uso	ripasso rapido, orale, relazione tecnica

Idea chiave: il Canny non si limita a cercare variazioni di intensità; prima riduce il rumore con una Gaussiana, poi stima il gradiente, assottiglia i bordi e infine usa soglie e connettività per conservare solo i contorni significativi.

Diagramma di flusso dell'algoritmo

La sequenza seguente riassume l'intera pipeline implementata.

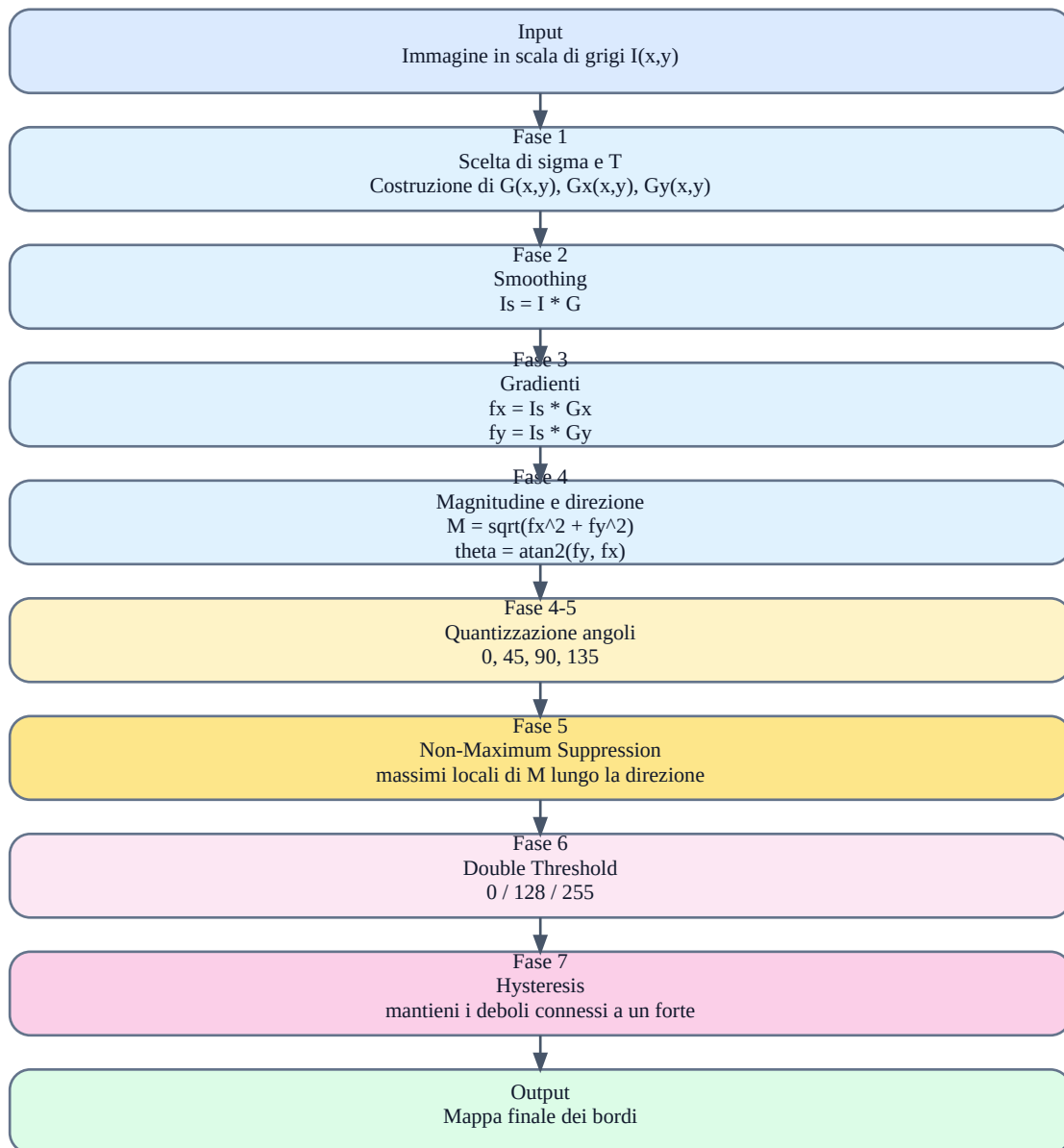


Figura 1 - Pipeline completa del Canny implementato.

1. Obiettivo generale

L'obiettivo del progetto è rilevare i bordi di un'immagine in modo robusto al rumore e con buona localizzazione spaziale. Un bordo è una zona in cui l'intensità luminosa cambia rapidamente. Il Canny cerca di massimizzare tre proprietà: buona rilevazione, buona localizzazione e risposta singola per bordo.

Nel tuo progetto l'algoritmo è stato implementato in modo esplicito: prima si costruiscono le maschere gaussiane e derivate, poi si applicano all'immagine, si calcolano magnitudine e direzione del gradiente, si quantizzano gli angoli, si esegue la Non-Maximum Suppression e infine si applicano doppia soglia e isteresi.

Convenzione sugli assi usata nel progetto:

- asse x associato alle righe dell'immagine, quindi orientato verticalmente verso il basso;
- asse y associato alle colonne dell'immagine, quindi orientato orizzontalmente verso destra.

2. Fase 1 - Costruzione delle maschere gaussiane e derivate

Questa fase genera il filtro di smoothing gaussiano e i due filtri derivativi Gx e Gy che verranno poi convoluti con l'immagine. La costruzione dipende da due parametri: sigma, che controlla la scala di smoothing, e T, che controlla il troncamento della Gaussiana e quindi la dimensione del kernel.

2.1 Scelta della dimensione del kernel

Gaussiana 1D troncata con soglia T:

$$\exp(-x^2 / (2 * \sigma^2)) = T$$

Da cui:

$$s_half = \text{round}(\sqrt{-\log(T) * 2 * \sigma^2})$$

$$\text{masksize} = 2 * s_half + 1$$

La Gaussiana teorica ha supporto infinito. In pratica si sceglie una soglia T tra 0 e 1 e si considera trascurabile il contributo oltre il punto in cui la Gaussiana scende sotto T. In questo modo il kernel è abbastanza grande da rappresentare il filtro ma non inutilmente costoso.

Interpretazione dei parametri

- sigma piccolo -> campana stretta, poco smoothing, maggiore sensibilità a dettagli e rumore;
- sigma grande -> campana larga, smoothing più forte, minore rumore ma possibile perdita di dettagli fini;
- T piccolo -> kernel più grande, perché si tronca la Gaussiana più lontano dal centro;
- T grande -> kernel più piccolo, perché si considera trascurabile il contributo già vicino al centro.

Nel progetto è stato anche imposto $\sigma \geq 0.5$ per evitare kernel troppo piccoli e poco significativi.

2.2 Generazione della griglia discreta

```
half = Half_mask(T, sigma)
```

```
valori = range(-half, half + 1)
```

```
y, x = np.meshgrid(valori, valori)
```

La funzione MaskGen costruisce due matrici coordinate centrate in 0. Ogni elemento del kernel sa quindi in quale posizione si trova rispetto al centro. Queste coordinate discrete saranno usate per valutare sia la Gaussiana sia le sue derivate.

2.3 Gaussiana 2D

$$\text{var} = (x^2 + y^2) / (2 * \sigma^2)$$

$$\begin{aligned} G(x, y) &= \exp(-\text{var}) \\ &= \exp(-(x^2 + y^2) / (2 * \sigma^2)) \end{aligned}$$

La formula usata nel progetto omette il fattore di normalizzazione $1 / (2 * \pi * \sigma^2)$, perché ai fini del filtraggio e del gradiente interessa soprattutto la forma del filtro. La Gaussiana ottenuta è una superficie a campana centrata in (0,0), con massimo al centro e decrescita radiale simmetrica.

2.4 Derivate della Gaussiana

$$G_x(x, y) = - (x / \sigma^2) * \exp(-(x^2 + y^2) / (2 * \sigma^2))$$

$$G_y(x, y) = - (y / \sigma^2) * \exp(-(x^2 + y^2) / (2 * \sigma^2))$$

Questi filtri misurano le variazioni locali di intensità nelle due direzioni. La struttura dei kernel è antisimmetrica: una parte negativa, un centro circa nullo e una parte positiva. Questa forma rende i filtri adatti a rispondere ai cambiamenti e non alle regioni uniformi.

2.5 Scaling e discretizzazione dei kernel derivativi

$$g_x = \text{round}(255 * G_x(x, y))$$

$$g_y = \text{round}(255 * G_y(x, y))$$

Lo scaling con 255 rende i valori del kernel più leggibili e stabili numericamente. Non altera la logica del gradiente: cambia la scala dei valori, non la direzione del vettore gradiente. Nel progetto è stato anche applicato un cambio di segno finale per uniformarsi a convenzioni più intuitive di visualizzazione.

3. Fase 2 - Smoothing e applicazione delle maschere

Una volta costruiti i kernel, il primo utilizzo effettivo è lo smoothing gaussiano, necessario per ridurre il rumore prima del calcolo delle derivate. Subito dopo si convolvono l'immagine smussata con G_x e G_y per ottenere le due componenti del gradiente.

3.1 Padding

$$\text{padded} = \text{matrice_di_zeri}(\text{righe} + k_{\text{righe}}, \text{colonne} + k_{\text{colonne}})$$

$$\text{padded}[\text{insert} : \text{insert} + \text{righe}, \text{insert} : \text{insert} + \text{colonne}] = \text{Img}$$

Il padding aggiunge un bordo artificiale attorno all'immagine per poter calcolare la convoluzione anche sui pixel al bordo. Nel progetto è stato usato zero padding.

3.2 Convoluzione di smoothing

$$I_s(x, y) = \sum_i \sum_j [G(i, j) * I_{\text{pad}}(x + i, y + j)] / \sum_i \sum_j G(i, j)$$

La divisione per la somma del kernel serve a normalizzare il risultato, mantenendo coerente l'intensità media dell'immagine. Questa operazione produce l'immagine smussata I_s .

3.3 Convoluzione con i kernel derivativi

$$f_x(x, y) = \sum_i \sum_j [G_{x_rot}(i, j) * I_{\text{pad}}(x + i, y + j)]$$

$$f_y(x, y) = \sum_i \sum_j [G_{y_rot}(i, j) * I_{\text{pad}}(x + i, y + j)]$$

Nel codice il kernel viene ruotato di 180 gradi con `np.fliplr` e `np.flipud` per realizzare una vera convoluzione e non una correlazione. Qui non si divide per la somma del kernel, perché i filtri derivativi non hanno funzione di media ma di differenza locale.

Significato di f_x e f_y

- f_x rappresenta la risposta del filtro derivativo associato alla direzione x ;

- f_y rappresenta la risposta del filtro derivativo associato alla direzione y ;
- il segno del valore indica il verso del cambiamento di intensità;
- il modulo del valore indica la forza del cambiamento.

4. Fase 3 - Magnitudine del gradiente

Le due immagini f_x e f_y vanno ora combinate in un'unica misura che indichi quanto e forte il bordo in ciascun punto.

4.1 Formula della magnitudine

$$M(x, y) = \sqrt{f_x(x, y)^2 + f_y(x, y)^2}$$

Questa è la lunghezza del vettore gradiente. Valori alti identificano punti in cui l'intensità cambia rapidamente. Valori bassi corrispondono invece a zone uniformi o poco variabili.

4.2 Normalizzazione usata nel progetto

$$mg = M * 100 / M_{max}$$

$$mg = \text{round}(mg)$$

La normalizzazione porta i valori su una scala 0-100 per rendere più semplice la visualizzazione e la scelta delle soglie nelle fasi successive.

5. Fase 4 - Direzione del gradiente

Oltre alla forza del bordo, il Canny ha bisogno della sua direzione. La direzione del gradiente indica il verso del massimo aumento di intensità ed è perpendicolare al bordo.

5.1 Formula della direzione

$$\theta(x, y) = \arctan2(f_y(x, y), f_x(x, y))$$

$$\theta_{deg} = \text{rad2deg}(\theta)$$

$$\theta_{final} = \theta_{deg} + 180$$

Nel progetto la direzione viene convertita in gradi e traslata di 180 gradi per lavorare nell'intervallo $[0, 360]$. La funzione $\arctan2$ è fondamentale per distinguere correttamente i quadranti e per evitare problemi quando $f_x = 0$.

Interpretazione geometrica

- il gradiente punta verso la zona più chiara;
- la sua direzione è ortogonale al bordo;
- direzioni opposte differiscono di 180 gradi ma, per il confronto dei massimi locali, appartengono alla stessa linea orientata.

6. Fase intermedia - Quantizzazione degli angoli

La direzione del gradiente è continua, ma nella Non-Maximum Suppression è sufficiente lavorare con 4 orientazioni principali. Per questo gli angoli vengono quantizzati.

Classe	Intervalli angolari	Direzione rappresentata	Vicini da confrontare nella NMS
0	0-22.5, 157.5-202.5, 337.5-360	0 gradi	sinistra e destra
1	22.5-67.5, 202.5-247.5	45 gradi	alto-destra e basso-sinistra
2	67.5-122.5, 247.5-292.5	90 gradi	sopra e sotto

3	122.5-157.5, 292.5-337.5	135 gradi	alto-sinistra e basso-destra
---	--------------------------	-----------	------------------------------

Scopo della quantizzazione: non serve per misurare meglio l'angolo, ma per decidere in modo semplice e stabile quali due pixel adiacenti vadano confrontati nella fase successiva.

7. Fase 5 - Non-Maximum Suppression

La magnitudine del gradiente è spesso alta su una banda di più pixel attorno al bordo. La Non-Maximum Suppression assottiglia questa banda, mantenendo solo i massimi locali lungo la direzione del gradiente.

7.1 Regola concettuale della NMS

Per ogni pixel (i, j) :

- 1) usa $\text{quantZ}(i, j)$ per scegliere i due vicini corretti;
- 2) confronta $M(i, j)$ con quei due vicini;
- 3) se $M(i, j) \geq$ entrambi, allora conserva $M(i, j)$;
- 4) altrimenti imposta il pixel a 0.

La direzione quantizzata serve solo a decidere dove guardare. Il confronto deve essere fatto sulla magnitudine del gradiente, non sull'angolo.

7.2 Forma corretta del confronto

```

se quantZ[i,j] == 0:
    confronta M[i,j] con M[i, j-1] e M[i, j+1]

se quantZ[i,j] == 1:
    confronta M[i,j] con M[i-1, j+1] e M[i+1, j-1]

se quantZ[i,j] == 2:
    confronta M[i,j] con M[i-1, j] e M[i+1, j]

se quantZ[i,j] == 3:
    confronta M[i,j] con M[i-1, j-1] e M[i+1, j+1]
```

Questo è il punto più importante da ricordare a ripasso: nella NMS la magnitudine decide chi sopravvive, la direzione decide solo con chi va confrontato il pixel corrente.

Nota di correzione importante

Nel codice iniziale il confronto era stato impostato usando la matrice degli angoli *Direction*. Concettualmente la NMS corretta deve invece confrontare la magnitudine *mg*. In altre parole: *Direction* dice in che direzione guardare, *mg* dice quale pixel è il massimo locale.

8. Fase 6 - Double Threshold

Dopo la NMS i bordi sono sottili, ma non tutti sono ugualmente affidabili. La doppia soglia distingue tra non bordo, bordo debole e bordo forte.

8.1 Regola della doppia soglia

```

se NMS(i, j) < low_threshold:
    output(i, j) = 0

se low_threshold <= NMS(i, j) < high_threshold:
    output(i, j) = 128

se NMS(i, j) >= high_threshold:
    output(i, j) = 255
```

Nel progetto i valori 0, 128 e 255 sono stati usati come etichette pratiche: 0 = non bordo, 128 = bordo debole, 255 = bordo forte.

Ruolo delle soglie

- soglia alta: accetta solo bordi molto credibili come forti;
- soglia bassa: evita di scartare subito bordi più deboli ma potenzialmente utili;
- valori troppo bassi -> troppo rumore;
- valori troppo alti -> perdita di dettagli e contorni interrotti.

9. Fase 7 - Hysteresis

L'isteresi è il passaggio che rende il Canny molto robusto: un bordo debole viene mantenuto solo se appartiene a una catena connessa a un bordo forte. In questo modo si eliminano i deboli isolati, tipicamente dovuti al rumore, ma si conservano i segmenti deboli che completano un contorno reale.

9.1 Regola dell'isteresi

```
se pixel == 255:  
    resta 255  
  
se pixel == 128:  
    diventa 255 solo se è connesso a un pixel 255  
    nella 8-neighborhood; altrimenti diventa 0
```

La 8-neighborhood comprende i vicini orizzontali, verticali e diagonali. La connettività non va vista solo come controllo immediato: i bordi deboli possono formare catene collegate indirettamente a un forte.

9.2 Osservazione implementativa importante

Una sola scansione non sempre basta.

Esempio:
255 128 128 128

Con una sola passata, solo il primo 128 diventerebbe 255.
Per ottenere il comportamento corretto bisogna propagare
la connessione iterativamente o tramite visita del grafo dei pixel.

Questa è la correzione principale emersa durante il ripasso: la tua prima versione dell'hysteresis promuoveva solo i deboli direttamente adiacenti a un forte, ma non propagava la promozione lungo catene di deboli. La versione corretta deve ripetere il controllo finché non ci sono più cambiamenti, oppure usare una struttura tipo stack o queue a partire dai pixel forti.

10. Riassunto finale della pipeline

Fase	Input	Output	Idea chiave
1. Kernel	sigma, T	G, Gx, Gy	costruire filtri coerenti con la scala scelta
2. Smoothing	I, G	Is	ridurre il rumore prima delle derivate
3. Gradienti	Is, Gx, Gy	fx, fy	stimare le variazioni di intensità
4. Magnitudine	fx, fy	M	misurare la forza del bordo
5. Direzione	fx, fy	theta	sapere lungo quale linea confrontare
6. Quantizzazione	theta	quantZ	ridurre a 4 orientazioni principali
7. NMS	M, quantZ	bordi sottili	tenere solo i massimi locali

8. Double Threshold	NMS	0 / 128 / 255	separare forte, debole, non bordo
9. Hysteresis	thresholded	mappa finale	conservare solo i deboli connessi a forti

Tre frasi da ricordare all'orale:

- La Gaussiana riduce il rumore prima del calcolo del gradiente, per evitare falsi bordi dovuti alle derivate su segnali rumorosi.
- La Non-Maximum Suppression usa la direzione del gradiente per scegliere i vicini, ma confronta la magnitudine per mantenere solo i massimi locali.
- L'hysteresis conserva i bordi deboli solo se connessi a bordi forti, garantendo continuità dei contorni e robustezza al rumore.

Conclusione: il tuo progetto riproduce in modo didattico l'intera logica del Canny. I punti più delicati da ripassare sono la costruzione dei kernel, la differenza tra convoluzione e correlazione, la NMS corretta e l'isteresi con propagazione della connettività.